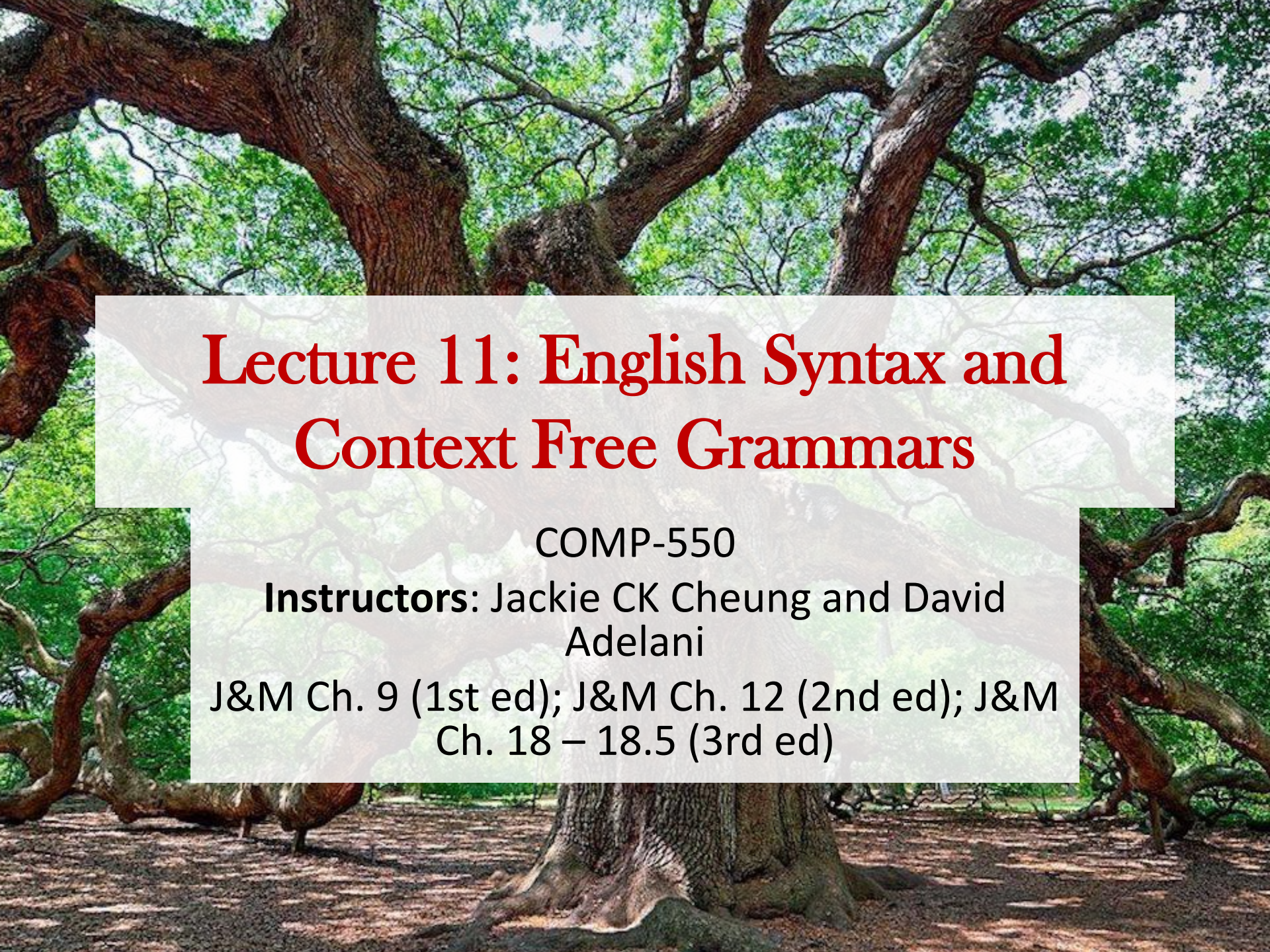




Lecture 11: English Syntax and Context Free Grammars

COMP-550

Instructors: Jackie CK Cheung and David
Adelani

J&M Ch. 9 (1st ed); J&M Ch. 12 (2nd ed); J&M
Ch. 18 – 18.5 (3rd ed)

Where Are We in the Course?

Passages as samples -> text classification **DONE**

Text as sequences -> sequence labelling **DONE**

Next: hierarchical structures!

Today's lecture: **syntax!**

- What is Syntax
- English Syntax
- Context Free Grammars

Language: Not Just A Sequence

Text is a discrete approximation of speech -> makes us think language is a sequence of tokens!

Consider:

- Sentence intonation patterns
 - The homework is due February 12.
 - The homework is due February 12?
 - The homework is due February 12!
- Liaison in French
 - En haut vs. en eau

Hierarchy in Language

More saliently, we can analyze the internals of sentences and find patterns that we can explain with **hierarchical structure**.

Today's lecture: provide evidence for this structure and a formal model of it!

Syntax

Study of how structures in a sentence can be assembled to form larger structures, recursively.

Key concern 1: what is a valid sentence? **Grammaticality**

- *This is a valid sentence.*
- **A sentence this valid is.*

An asterisk indicates **ungrammaticality**, by convention.

Want to specify all and exactly those sentences of a language which are grammatical

Key concern 2: How to map from syntactic structures and relationships to semantic structures and relationships?

The First Grammarian

Panini (Pāṇini) from the 4th century B.C. developed a grammar for Sanskrit.

51. The affixes ktvâ, क्त and क्तवतु optionally get इट् after पू ॥

As पूत्वा or पवित्र्वा, सोमोत्तिपूतः, सोमोत्तिपवितः पूतवान् or पवितवान् ॥ This allows option where by VII. 2. 11 there would have been prohibition. See I. 2. 22.

वसतिश्चुधोरिट् ॥ ५२ ॥ पदानि ॥ वसति, चुधोः, इट् ॥

वृत्तिः ॥ वसतेः चुधेश्च क्तवानिष्ठयोरिडागमो भवति ।

52. The affix ktvâ, क्ता and क्तवतु always receive the augment इट् after वस् (वसति) and चुध् ॥

As उषित्वा, उषितः and उषितवान्, चुधित्वा, चुधितः, चुधितवान् ॥ The वस् of the Adâdi class will get इट् as it is enumerated in the list of सेट् roots. The repetition of इट् shows that the rule is invariable, the 'optionally' of the preceding sūtra does not affect it.

अञ्चेः पूजायाम् ॥ ५३ ॥ पदानि ॥ अञ्चेः, पूजायाम् ॥

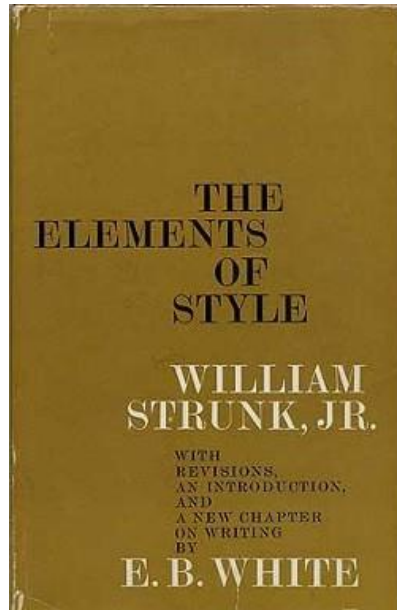
वृत्तिः ॥ अञ्चेः पूजायामर्थे क्तवानिष्ठयोरिडागमो भवति ।

Source: <https://archive.org/details/ashtadhyayitrans06paniuoft>

What We Don't Mean by Grammar

Rules or guides for how to write properly

e.g.,



These style guides are **prescriptive**. We are concerned with **descriptive** grammars of naturally occurring language.

Hierarchical Structure of Language

How is language hierarchical? Via **constituents**.

Constituents: A group of words that behave as a unit

Noun phrases:

- *computational linguistics, it, Justin Trudeau, three people on the bus, “Jean-Claude Van Damme, the Muscles from Brussels”*

Adjective phrases:

- *blue, purple, very good, ridiculously annoying and tame*

How do these words form a unit? They share similar distributional properties which we can test for.

Tests for Constituency

1. They can appear in similar syntactic environments.

I saw ...

it

Jean-Claude Van Damme, the Muscles from Brussels

three people on the bus

**Van*

**on the*

Tests for Constituency

2. They can be placed in different positions or replaced in a sentence as a unit.

[Jean-Claude Van Damme, the Muscles from Brussels], beat me up.

It was [Jean-Claude Van Damme, the Muscles from Brussels], who beat me up.

I was beaten up by [Jean-Claude Van Damme, the Muscles from Brussels].

He beat me up. (i.e., J-C V D, the M from B)

Tests for Constituency

3. It can be used to answer a question.

Who beat you up?

[Jean-Claude Van Damme, the Muscles from Brussels]

**[the Muscles from]*

The type of a constituent is called its **syntactic category**

e.g., verb phrases, noun phrases, adjective phrases,
prepositional phrases, clauses, sentences, etc.

Grammatical Relations

Relationships between different constituents. Here are some related to verbs:

Subject

- *Jean-Claude Van Damme relaxed.*
- *The wallet was stolen by a thief.*

(Direct) object

- *The boy kicked the ball.*

Indirect object

- *She gave him a good beating.*

There are many other grammatical relations.

Subcategorization

Verbs (and other words) require a different number of **arguments**:

relax	1	subj
steal*	2	subj, dobj
kick	2	subj, dobj
give	3	subj, iobj, dobj

*the passive changes the subcategorization of the verb

Think of a parallel in programming language: functions also take a certain number of arguments

- multiply(x, y) might multiply two numbers x and y

More Subcategorization

Some other possibilities:

want 2 subj, inf. clause

- *I want to learn about computational linguistics.*

apprise 3 subj, obj, pobj with of

- *The minister apprised him of the new developments.*

different 2 subj, pobj with from/than/to

- *This course is different [from/than/to] what I expected.*

Exercise

Identify the prepositional phrase in the following sentence. Give arguments for why it is a constituent.

The next assignment is due on Thursday, February 12th.

Formal Grammars

As computational linguists, we will use a formal computational model of grammar to account for these and other syntactic concerns.

Formal grammar

Rules that generate a set of strings that make up a **language**.

(In this context, language simply refers to a set of strings.)

Why?

- Formal understanding lets us develop appropriate algorithms for dealing with syntax.
- Implications for cognitive science/language learning

FSAs and Regular Grammars

You may have encountered finite-state machines/regular languages before

- An FSA generates a **regular language**
- FSAs correspond to a class of formal grammars called **regular grammars**
- Used for NLP tasks such as stemming, lemmatization, and morphological analysis

To describe the syntax of natural languages (with multiple constituents, subcategorization, etc.), we need a more powerful class of formal grammars – **context free grammars (CFGs)**.

Context Free Grammars (CFG)s

Rules that describe what possible sentences are:

$S \rightarrow NP VP$

$NP \rightarrow \text{this}$

$VP \rightarrow V$

$V \rightarrow \text{is} \mid \text{kicks} \mid \text{jumps} \mid \text{rocks}$

Constituent Tree

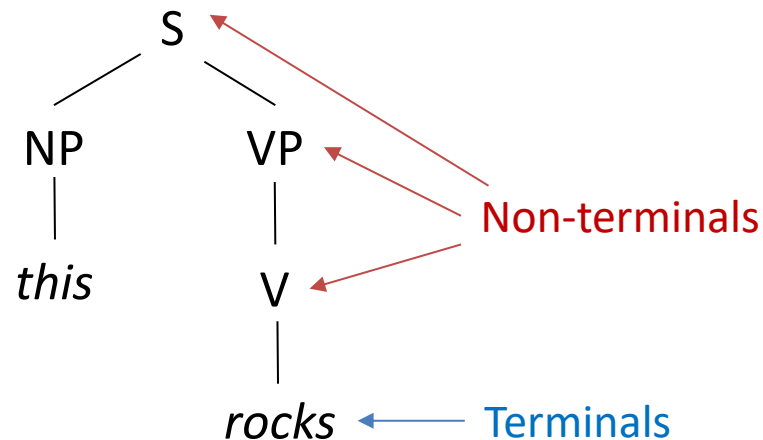
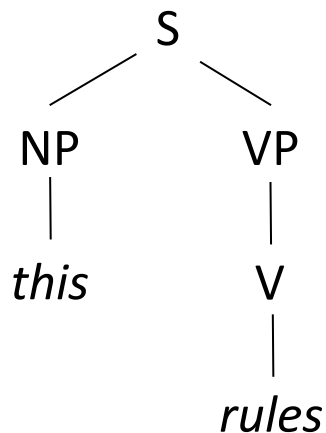
Trees (and sentences) generated by the previous rules:

$S \rightarrow NP VP$

$NP \rightarrow this$

$VP \rightarrow V$

$V \rightarrow is \mid rules \mid jumps \mid rocks$



Formal Definition of a CFG

A 4-tuple:

N set of **non-terminal** symbols

Σ set of **terminal** symbols

R set of **rules** or **productions** in the form $A \rightarrow (\Sigma \cup N)^*$,
and $A \in N$

S a designated **start symbol**, $S \in N$

Extended Example

Let's develop a CFG that can account for verbs with different subcategorization frames:

intransitive verbs	<i>relax</i>	1	subj
transitive verbs	<i>steal, kick</i>	2	subj, dobj
ditransitive verbs	<i>give</i>	3	subj, iobj, dobj

Undergeneration and Overgeneration

Problems with above grammar:

Undergeneration: misses valid English sentences

- *The boy kicked the ball softly.*
- *The thief stole the wallet with ease.*

Overgeneration: generates ungrammatical sentences

- **The boy kick the ball.*
- **The thieves steals the wallets.*

Extension 1

Let's add adverbs and prepositional phrases to our grammar

Recursion

Consider the following sentences:

- *The dog barked.*
- *I know that the dog barked.*
- *You know that I know that the dog barked.*
- *He knows that you know that I know that the dog barked.*
- ...

In general:

S -> NP VP

VP -> V_{intr}

V_{intr} -> *barked*

VP -> V_{that} S_{that}

V_{that} -> *know*

S_{that} -> *that S*

Recursion

This recursion in the syntax of English means that sentences can be infinitely long (theoretically).

- For a given sentence S , you can always make it longer by adding [I/you/he know(s) that S].

In practice, the length is limited because we have limited attention span/memory/processing power.

Exercise

Let's try to fix the subject-verb agreement issue:

Present tense:

Singular third-person subject -> verb has affix of -s or -es

Otherwise -> base form of verb

(*to be* is an exception, along with other irregular verbs)

Dependency Grammar

Grammatical relations induce a **dependency relation** between the words that are involved.

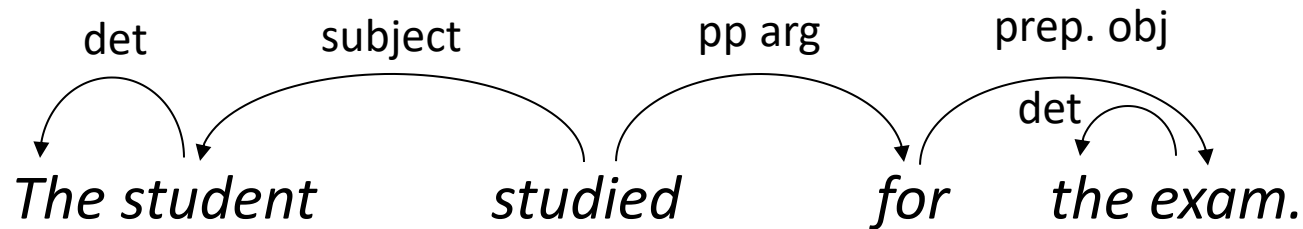
The student studied for the exam.

Each phrase has a **head word**.

- the student studied for the exam
- the student
- for the exam
- the exam

Dependency Grammar

We can represent the grammatical relations between phrases as directed edges between their heads.



This lets us get at the relationships between words and phrases in the sentence more easily.

Who/what are involved in the studying event?

- student, for the exam

Converting between Formalisms

Dependency trees can be converted into a standard constituent tree deterministically (if the dependency edges don't cross each other).

Constituent trees can be converted into a dependency tree, if you know what is the **head** of the constituent.

Let's convert some of our previous examples...

Crossing Dependencies

Dependencies can cross.

Especially if the language has **freer word order**:

Er hat mich versucht zu erreichen.

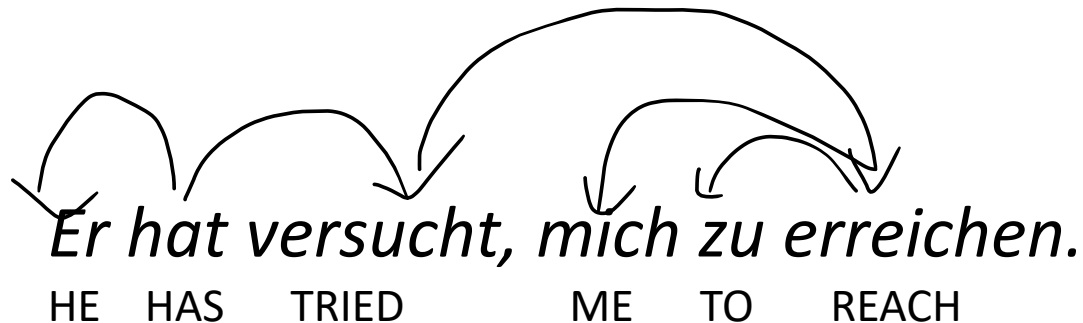
Er hat versucht mich zu erreichen.

He tried to reach me.

These have the same literal meaning.

Crossing Dependencies Example

What would the dependency edges be in these cases?



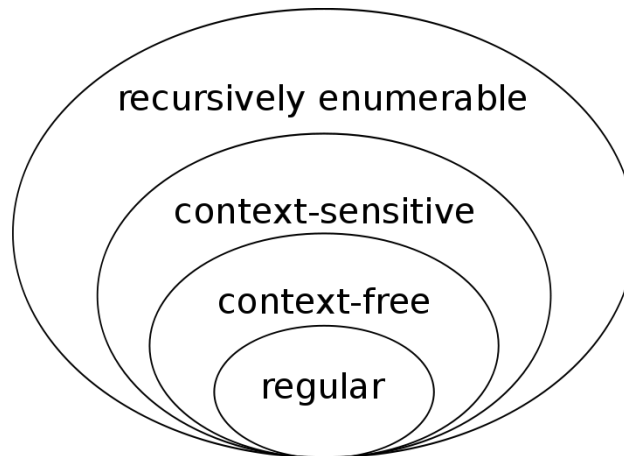
Notice the discontinuous constituent that results in the second case.

Are Natural Languages CFGs?

Recall that a formal language is defined to be a set of strings constructed over a specified vocabulary

Are natural languages CFGs? i.e., can we define each natural language (e.g., English, French, German, etc.) as a CFG?

Other possibilities: **Chomsky hierarchy**



https://en.wikipedia.org/wiki/Chomsky_hierarchy

Cross-serial Dependencies

Swiss German (Shieber, 1985) and Bambara (Culy, 1985) have structures that generate strings which cannot be captured by CFGs (**cross-serial dependencies**):

$$a^m b^n c^m d^n$$

Relies on following assumption:

- m and n can be arbitrarily large values
- strings are either in a language or not (grammatical or ungrammatical)

May not be the most useful question to ask after all

Parsing

Input sentence, grammar $\rightarrow$ output parse tree

Parsing into a CFG: **constituent parsing**

Parsing into a dependency representation: **dependency parsing**

Difficulty: need an efficient way to search through plausible parse trees for the input sentence

Parsing into a CFG

Given:

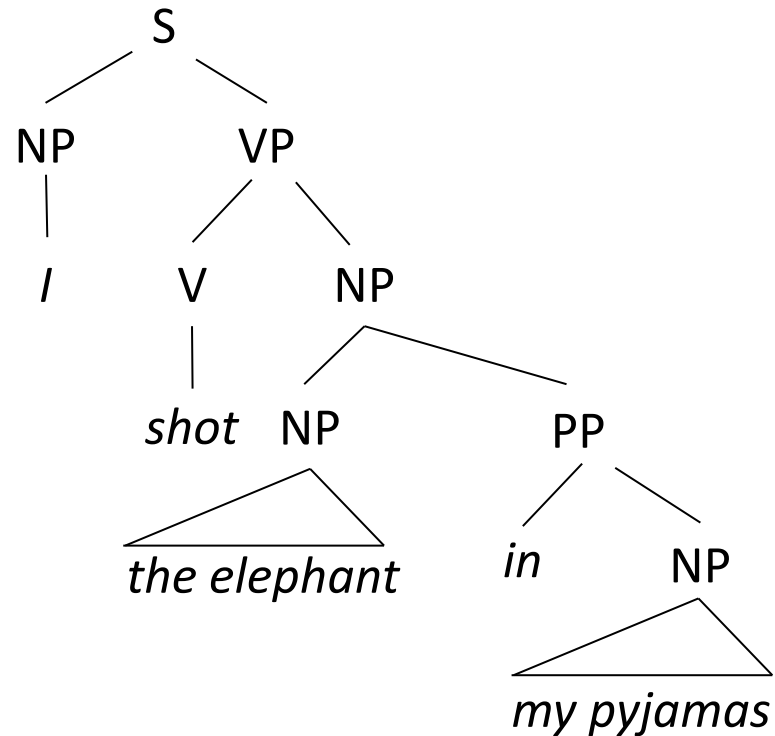
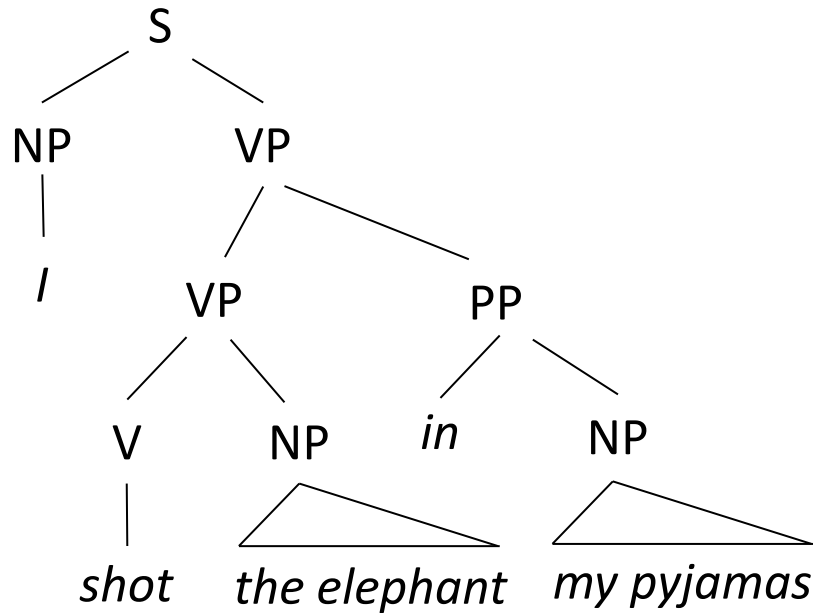
1. CFG
2. A sentence made up of words that are in the terminal vocabulary of the CFG

Task: Recover all possible parses of the sentence.

Why *all* possible parses?

Syntactic Ambiguity

I shot the elephant in my pyjamas.



Types of Parsing Algorithms

Top-down

Start at the top of the tree, and expand downwards by using rewrite rules of the CFG to match the tokens in the input string

e.g., Earley parser

Bottom-up

Start from the input words, and build ever-bigger subtrees, until a tree that spans the whole sentence is found

e.g., **CYK algorithm**, shift-reduce parser

Key to efficiency is to have an efficient search strategy that avoids redundant computation

CYK Algorithm

Cocke-Younger-Kasami algorithm

- A **dynamic programming** algorithm – partial solutions are stored and efficiently reused to find all possible parses for the entire sentence.
- Also known as the CKY algorithm

Steps:

1. Convert CFG to an appropriate form
2. Set up a table of possible constituents
3. Fill in table
4. Read table to recover all possible parses